NEXUS — Manual técnico y preparación para el examen H3

I. Arquitectura y programación orientada a objetos

1. Propósito y funcionamiento general de NEXUS

NEXUS es un proyecto desarrollado en C# que simula una aplicación interactiva y juego de consola. El objetivo central del programa es administrar el flujo de ejecución del juego, mantener actualizado el estado de sus componentes y gestionar las interacciones con el usuario empleando estructuras de control y manejo de datos en consola.

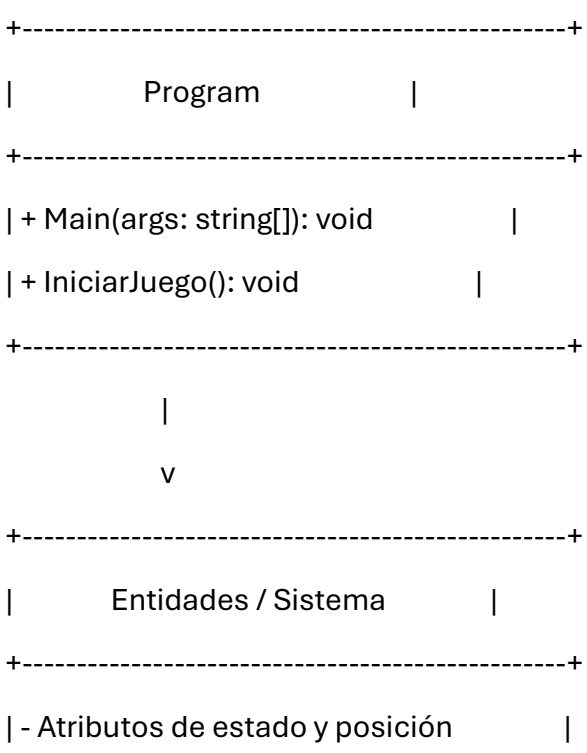
2. Estructura del proyecto y responsabilidades de sus clases

Toda la estructura principal del programa se encuentra organizada dentro de Program.cs, distribuyendo las tareas de la siguiente manera:

- **Clase Principal (Program):** Es el punto de partida del ejecutable (contiene el método Main). Se encarga de controlar el ciclo de vida de la aplicación, inicializar las variables de estado y desplegar el menú de interacción.
- **Clases de Entidad / Lógica:** Son las encargadas de empaquetar los atributos y las acciones del sistema, administrando aspectos como el jugador, la matriz o mapa del juego y los elementos del entorno.

3. Diagrama de clases y relaciones

Plaintext



| + Métodos de procesamiento y lógica |

+---+

4. Programación Estructurada vs. Programación Orientada a Objetos

- **Programación Estructurada:** Mantiene las funciones y las variables separadas. La lógica avanza paso a paso de forma secuencial, lo cual complica la lectura y el mantenimiento a medida que la aplicación crece.
- **Programación Orientada a Objetos (POO):** Agrupa en una sola unidad (clase) tanto la información (atributos) como el comportamiento (métodos). Esto facilita reutilizar código, proteger los datos internos mediante encapsulamiento y organizar el programa de forma modular para corregir fallas o agregar nuevas funciones.

II. Fundamentos de clases y objetos

1. Clases, objetos e instanciación

- **Clase:** Es la plantilla o molde donde definimos qué características y acciones tendrán los elementos de NEXUS.
- **Objeto:** Es la entidad real que creamos en la memoria RAM a partir de una clase mientras el programa está en ejecución.
- **Instanciación:** Es el procedimiento con el que reservamos espacio en memoria para crear un objeto usando la instrucción new:

C#

```
// Ejemplo de instanciación en NEXUS

MiObjeto objeto = new MiObjeto();
```

2. Atributos, campos, propiedades y métodos

- **Campos (Fields):** Variables internas declaradas en la clase que guardan el estado de forma privada.
- **Propiedades (Properties):** Bloques con descriptores get y set que nos permiten leer o modificar datos de manera controlada y validada.
- **Métodos (Methods):** Funciones de la clase que definen lo que el objeto puede hacer.

3. Constructores y sobrecarga

- **Constructor:** Método especial que se llama automáticamente al crear una instancia para darle valores iniciales al objeto.

- **Sobrecarga (Overloading):** Capacidad de tener más de un constructor o método con el mismo nombre pero cambiando sus parámetros (ya sea en tipo o cantidad), lo que nos da flexibilidad al momento de instanciar objetos.

4. Encapsulamiento y modificadores de acceso

El encapsulamiento protege la información interna para que no sea alterada sin control. En NEXUS usamos los siguientes modificadores:

- private: El elemento solo puede usarse dentro de la clase que lo declaró.
- public: El elemento puede consultarse o modificarse desde cualquier parte del proyecto.
- protected: Accesible dentro de la misma clase y en sus clases derivadas a través de la herencia.

5. Uso de this

- La palabra reservada this se refiere a la instancia concreta sobre la que se está ejecutando el código. Nos ayuda a evitar confusiones cuando un parámetro tiene el mismo nombre que un atributo de la clase.

III. Herencia, polimorfismo e interfaces

1. Clases base y clases derivadas

- **Clase base (Padre):** Agrupa las propiedades y métodos generales que van a compartir distintos elementos de nuestro sistema.
- **Clase derivada (Hija):** Hereda lo definido en la clase base usando la sintaxis de dos puntos (:) en C#, lo que permite reutilizar lógica y agregar funcionalidades específicas.

2. Reutilización y especialización de comportamientos

Gracias a la herencia no tenemos que volver a escribir código repetido. Permite reutilizar lo que ya funciona bien en la clase base y especializar a las clases hijas para que reaccionen según lo necesite el programa.

3. Métodos virtual y override

- virtual: Indica en la clase base que un método puede ser redefinido en sus clases hijas.
- override: Se coloca en la clase derivada para modificar deliberadamente la implementación original del método base.

4. Clases y métodos abstract

- **Clase abstracta (abstract):** Sirve como una plantilla de diseño que no se puede instanciar directamente mediante new.
- **Método abstracto:** Es un método declarado sin cuerpo en la clase base que obliga a todas las clases hijas a definir su propia versión.

5. Polimorfismo y resolución de llamadas

El polimorfismo nos da la posibilidad de tratar a objetos de distintas clases derivadas usando la referencia de su clase base común. Al ejecutar la aplicación, C# determina en tiempo de ejecución cuál es el método real que le corresponde a la instancia actual.

6. Interfaces y contratos

Una **Interfaz** (interface) es un contrato formal donde indicamos qué métodos o propiedades debe implementar una clase, promoviendo un diseño flexible y con poco acoplamiento.

7. Diferencias entre herencia, composición e implementación de interfaces

- **Herencia ("Es un"):** Es una relación rígida entre clases (ej. *Jugador es un Personaje*). C# solo soporta herencia simple.
- **Composición ("Tiene un"):** Ocurre cuando una clase utiliza instancias de otras clases como parte de sus atributos internos para cumplir una tarea.
- **Implementación de Interfaz ("Cumple con"):** Asegura que una clase contenga las acciones requeridas por el contrato, permitiendo implementar múltiples interfaces al mismo tiempo.

IV. Colecciones y LINQ

1. Listas genéricas List<T>

- Las colecciones como List<T> nos permiten guardar datos de manera dinámica sin tener que definir un tamaño fijo como ocurre con los arreglos tradicionales. Esto facilita agregar, eliminar o buscar objetos durante la ejecución de NEXUS.

2. Manipulación y recorrido de objetos

- Para recorrer estas colecciones en C# la opción estándar es el bucle foreach. Es una forma clara y segura de revisar cada objeto almacenado para leer sus propiedades o llamar a sus métodos.

3. Expresiones lambda

- Son funciones anónimas resumidas que utilizan el operador =>. Se combinan con LINQ para escribir condiciones de filtro o criterios de orden de forma directa (por ejemplo: x => x.Id == 1).

4. Consultas LINQ (Where, Select, OrderBy)

LINQ (*Language Integrated Query*) nos permite consultar colecciones en C# mediante código declarativo:

- Where: Filtra la lista según una regla o condición lógica.
- Select: Proyecta o transforma los datos para extraer ciertos campos o convertirlos.
- OrderBy / OrderByDescending: Ordena los elementos de manera ascendente o descendente según el criterio indicado.

5. Comparación entre consultas LINQ y recorridos tradicionales

- **Búcles tradicionales (for / foreach con if anidados):** Requieren escribir variables auxiliares y controlar manualmente los pasos del recorrido, aumentando la cantidad de líneas de código.
- **Consultas LINQ:** Se enfocan en indicar *qué* datos queremos extraer en lugar de detallar *cómo* buscarlos paso a paso, dando como resultado un código más ordenado y fácil de mantener.

V. Análisis del funcionamiento

1. Flujograma de un proceso importante de NEXUS

Plaintext

[Inicio del Programa]

    |

    v

[Inicialización de Variables y Objetos (Listas, Instancias)]

    |

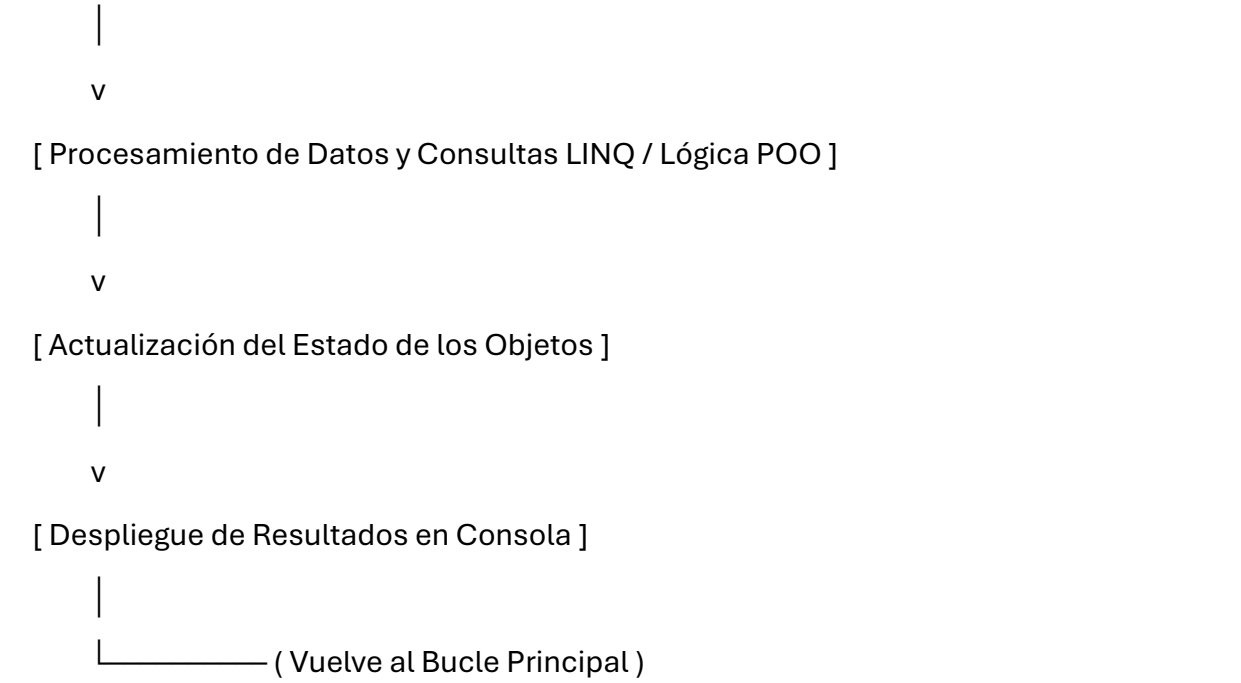
    v

[Bucle Principal / Menú de Interacción] —> (¿Condición de salida?) — Sí —> [Fin]

    | No

    v

[Captura de Entrada del Usuario]



2. Seguimiento del estado de un objeto durante la ejecución

- **Escenario de prueba:** Supongamos una entidad con la propiedad de puntuación Puntos.
    - **Estado inicial:** Al instanciar la clase mediante el constructor, Puntos = 0.
    - **Ejecución del método:** Al avanzar en la partida, se invoca un método de incremento: miObjeto.SumarPuntos(10);.
    - **Estado intermedio:** El atributo interno del objeto cambia su valor a 10.
    - **Estado final:** Tras procesar la acción con LINQ o evaluarla, el dato se confirma dentro de la lista global del programa.

3. Predicción del resultado de un fragmento de código

- **Código de análisis:**

C#

```
List<int> numeros = new List<int> { 5, 12, 3, 8, 20 };

var resultado = numeros.Where(n => n > 10).OrderBy(n => n).ToList();
```

- **Evaluación paso a paso:**
    1. El filtro .Where(n => n > 10) extrae los valores estrictamente mayores a 10: [12, 20].

2. La función .OrderBy(n => n) los ordena de forma ascendente, conservando el orden [12, 20].
3. **Resultado final:** La lista devuelta contendrá los elementos **[12, 20]**.

4. Caso de modificación de código

- **Pregunta:** ¿Qué ocurre si cambiamos un bucle foreach por una consulta LINQ usando .Select() para transformar elementos?
- **Análisis:** En términos de lectura obtendríamos un código más limpio y declarativo. Sin embargo, si la función dentro de .Select() pretende modificar directamente atributos de los objetos de la lista original, se rompe la inmutabilidad que se espera de las consultas puras en LINQ. Por ello, si el objetivo es alterar el estado interno de las instancias, es más adecuado usar un bucle tradicional o la función .ForEach().

VI. Preparación para el examen

1. Comprensión de POO

- **Pregunta:** ¿Qué diferencia fundamental existe entre la programación estructurada y la Programación Orientada a Objetos al desarrollar una aplicación en C#?
- **Respuesta razonada:** La programación estructurada trabaja con funciones y variables separadas de forma secuencial, dificultando el mantenimiento a gran escala. La POO agrupa los datos (atributos) y el comportamiento (métodos) en clases cohesionadas, lo que permite modelar entidades del mundo real, reutilizar código con herencia y proteger la integridad del estado mediante el encapsulamiento.

2. Análisis de clases y objetos

- **Pregunta:** ¿Por qué no debemos declarar los campos de una clase como public y qué beneficio ofrecen las propiedades con get y set?
- **Respuesta razonada:** Dejar los campos como públicos rompe el encapsulamiento, exponiendo los datos a modificaciones indebidas desde cualquier parte del código. Las propiedades con get y set crean un canal controlado donde podemos validar la información antes de guardarla o limitar los permisos de acceso.

3. Herencia y polimorfismo

- **Pregunta:** ¿Qué rol cumplen las instrucciones virtual y override cuando trabajamos con polimorfismo?

- **Respuesta razonada:** La etiqueta virtual le indica al compilador que un método en la clase base puede reescribirse, mientras que override nos permite definir la nueva lógica en la clase derivada. Gracias al polimorfismo, el entorno de C# evalúa durante la ejecución qué clase concreta se instanció y ejecuta el método correspondiente.

4. Interfaces y diseño

- **Pregunta:** Sabiendo que C# solo admite herencia simple entre clases, ¿de qué forma podemos lograr que un objeto cumpla con múltiples responsabilidades?
- **Respuesta razonada:** Usando **Interfaces**. Como una interfaz es un contrato de comportamientos que la clase se compromete a implementar, una misma clase en C# puede implementar múltiples interfaces, logrando modularidad sin las complicaciones de la herencia múltiple.

5. Listas y LINQ

- **Pregunta:** ¿Qué ventajas nos da realizar consultas con LINQ (Where, Select) en lugar de usar ciclos for e if anidados?
- **Respuesta razonada:** LINQ nos permite redactar código declarativo que expresa qué datos queremos consultar. Esto disminuye la cantidad de líneas, evita errores con los límites de los índices de los arreglos y hace que el código sea fácil de leer y mantener.

6. Predicción, corrección o modificación de código

- **Pregunta:** ¿Qué beneficio aporta a la arquitectura de NEXUS que un método reciba una interfaz como parámetro en lugar de una clase concreta?
- **Respuesta razonada:** Genera un **bajo acoplamiento**. Al depender de una interfaz (abstracción), podemos reemplazar o actualizar la clase concreta por otra que cumpla el mismo contrato sin tener que modificar la lógica interna del método que la consume.

Diagnóstico del Proceso de Aprendizaje — Misión NEXUS

1. Fortalezas demostradas

- **Gestión práctica y operativa:** Demostraste habilidad para interactuar con herramientas de desarrollo y control de versiones, logrando inicializar el repositorio, subir correctamente los archivos clave del código fuente (`Program.cs` y el archivo `.csproj`) a través de la interfaz web de GitHub y omitiendo de manera adecuada las carpetas temporales (`bin` y `obj`).
- **Orientación a metas y entregables:** Mantuviste un ritmo de trabajo constante y secuencial mediante preguntas de avance directo ("que sigue ahora", "listo que prosigue"), asegurando el cumplimiento de los pasos requeridos para completar la documentación del proyecto académico de "Programación II".
- **Criterio de presentación:** Mostraste interés en cuidar la calidad formal de tu entrega académica al buscar alternativas para lograr un tono de redacción natural y universitario en el manual técnico.

2. Falencias observadas

- **Ausencia de producción teórica autónoma en el chat:** Durante esta sesión no se registraron aportes, explicaciones conceptuales o fragmentos de código redactados por iniciativa propia sobre los pilares de la Programación Orientada a Objetos o LINQ; la redacción y síntesis conceptual del manual técnico fue delegada y generada casi en su totalidad por el asistente.
- **Dependencia de la guía paso a paso:** Requeriste acompañamiento detallado e instruccional para realizar las acciones básicas en la plataforma de GitHub y en la estructura de los archivos.
- **Búsqueda inicial de soluciones automatizadas:** Consultaste por herramientas externas de "humanización" automática de texto antes de priorizar la comprensión analítica del contenido técnico de tu propia materia.

3. Diagnóstico del Hito 3

- **POO, clases y objetos:** *Evidencia insuficiente.* No se registró en la conversación ninguna explicación, definición o análisis propio por parte del usuario sobre la creación o instanciación de clases más allá de la ejecución práctica del archivo ya programado.
- **Constructores y sobrecarga:** *Evidencia insuficiente.* No hubo aportes ni consultas del estudiante que evidencien un razonamiento sobre la implementación de constructores en este chat.
- **Encapsulamiento y modificadores de acceso:** *Evidencia insuficiente.* El usuario no emitió razonamientos ni ejemplos sobre el uso de `private`, `public` o `protected`.

- **Herencia y polimorfismo:** *Evidencia insuficiente.* No se documentaron explicaciones del estudiante sobre jerarquías de clases, métodos `virtual`/`override` ni polimorfismo.
- **Interfaces:** *Evidencia insuficiente.* El concepto de contratos e interfaces no fue abordado o explicado de manera autónoma por el estudiante en la conversación.
- **Listas genéricas y LINQ:** *Evidencia insuficiente / Comprensión parcial indirecta.* Si bien el usuario subió y gestionó el proyecto NEXUS (el cual contiene la lógica de programación evaluada en la materia), en este canal de chat no se evidenció un desglose, análisis o discusión activa de cómo funcionan sintácticamente las listas `List<T>` o los métodos LINQ (`Where`, `Select`, `OrderBy`) por parte del estudiante.

Nota de justificación: Dado que la dinámica de la conversación se centró en la ejecución de pasos operativos en GitHub y en solicitar la redacción y pulido completo del manual técnico al asistente, el volumen de producción textual autónoma sobre la teoría de la materia es nulo, lo que deja a la mayoría de los conceptos clasificados como evidencia insuficiente dentro de este intercambio específico.

4. Mi forma de aprender

Tu estilo de aprendizaje durante esta actividad es **práctico, operativo y guiado por metas**. Te enfocas en resolver los requerimientos paso a paso mediante instrucciones directas y visuales (apoyándote en capturas de pantalla para validar tus acciones en entornos como GitHub). Prefieres delegar la estructuración de la documentación teórica en el asistente para concentrarte en la ejecución de las tareas requeridas por tu docente, manteniendo el control de las entregas finales.

5. Plan de estudio (Conceptos prioritarios a reforzar)

Para garantizar un dominio real de cara al examen del Hito 3, debes reforzar de forma prioritaria los siguientes 5 conocimientos mediante práctica independiente:

1. **Encapsulamiento y propiedades (`get`/`set`):**
   - *Ejercicio práctico:* Crea una clase `Nave` con atributos privados (`vida`, `combustible`) y escribe propiedades públicas con validaciones en el descriptor `set` para evitar valores negativos.
2. **Herencia y sobreescritura (`virtual` / `override`):**
   - *Ejercicio práctico:* Implementa una clase base `Personaje` con un método virtual `Mover()`, y hereda dos clases (`Enemigo` y `Jugador`) que sobrescriban el método con comportamientos distintos.
3. **Contratos mediante Interfaces:**
   - *Ejercicio práctico:* Define una interfaz `IAtacable` que obligue a implementar un método `RecibirDanio(int cantidad)`, y haz que dos clases sin relación de parentesco la utilicen.
4. **Manipulación de Listas Genéricas (`List<T>`):**

    - *Ejercicio práctico:* Declara una lista de objetos en una consola de C#, añade elementos dinámicamente y recórrelos utilizando un bucle `foreach` para mostrar sus datos en pantalla.
5. **Consultas LINQ (`Where`, `Select`, `OrderBy`):**
    - *Ejercicio práctico:* A partir de una lista de números o puntajes, escribe una consulta LINQ que filtre los valores mayores a un parámetro dado, los ordene de forma descendente y proyecte el resultado en una nueva lista.

6. Conclusión

Durante esta actividad, has demostrado competencia en el manejo operativo de herramientas de control de versiones y en la organización de los entregables de tu proyecto en la web. Sin embargo, a nivel conceptual del Hito 3, **no se generó evidencia directa de aprendizaje autónomo** en esta conversación, ya que las explicaciones técnicas, definiciones de POO y consultas LINQ fueron redactadas y estructuradas por el asistente a petición tuya.